

BOUYZOURAN Inès

FENICH ELFENICHE Leila

STARON Florentin

MI2-A



Cahier des charges: FLIPTECH

1. Présentation du projet et règles du jeu

Le FLIPTECH est un jeu de cartes inspiré du principe du jeu FLIP7. Il s'agit d'un jeu multijoueur où à chaque tour chaque joueur tente d'accumuler le plus de points possible en piochant des cartes, tout en prenant le risque de perdre les points de sa manche en cas de doublon. Le jeu repose donc sur un équilibre entre prise de risque et stratégie d'arrêt, en effet, à chaque tour, un joueur peut soit continuer de piocher pour augmenter son score, soit s'arrêter afin de sécuriser les points déjà obtenus durant la manche.

Une partie commence par l'enregistrement des joueurs, suivi du choix du type de pioche, une classique prédéfinie ou une personnalisée construite selon les choix des utilisateurs. La pioche est ensuite mélangée et la partie débute sous forme de manches successives.

Durant une manche, chaque joueur joue à son tour et pioche des cartes une par une. Chaque carte tirée est immédiatement appliquée au score du joueur et peut avoir des effets spécifiques selon sa valeur, notamment pour les cartes bonus. Le joueur doit également surveiller l'apparition de doublons dans sa main, en effet, si une carte identique est tirée deux fois, il perd immédiatement les points accumulés durant la manche. À l'inverse, s'il parvient à obtenir sept cartes différentes sans doublon, il réalise un "FLIP 7" qui lui apporte un bonus de 15 points et met fin à sa manche.

Une manche se termine lorsqu'un joueur décide de s'arrêter, lorsqu'il perd à cause d'un doublon, lorsqu'il atteint un FLIP 7 ou lorsque la pioche est épuisée. Les points obtenus à chaque manche sont cumulés dans un score total qui permet de suivre l'évolution des joueurs sur l'ensemble de la partie. La partie se termine lorsqu'un joueur atteint les 200 points ou lorsque la pioche est entièrement vidée. À la fin de la partie, les joueurs peuvent décider s'ils veulent ou non un fichier avec le classement final.

2. Organisation générale du programme

main.c

Le fichier principal main.c gère le déroulement global du jeu, il utilise une boucle principale où se trouvent des manches, la gestion des joueurs, le choix de la pioche ainsi que les conditions de fin de partie.

pioche.c

Le module pioche.c est responsable de la création et de la gestion du paquet de cartes, qu'il soit classique ou personnalisé, ainsi que du mélange et du tirage des cartes. La pioche classique est composée de 79 cartes classiques (de sorte qu'il y ait 1 carte 0, 1 carte 1, 2 cartes 2, 3 cartes 3, etc...) ainsi que de 6 cartes bonus qui sont : x2, +2, +4, +6, +8 et le +10, donc 85 cartes au total. Pour ce qui est de la pioche spéciale, il s'agit d'une amélioration possible. En effet, il s'agit d'une pioche personnalisée qui permet au joueur de saisir une quantité souhaitée pour chaque carte classique. Les bonus, quant à eux, ne changent pas de quantité, il y en a toujours 6. En début de partie, le joueur peut choisir entre 1 une pioche classique ou 2 une pioche spéciale. Il y a donc les fonctions : initialiser_pioche pour la pioche classique, piocheSpecial pour la pioche spéciale, melange_pioche qui mélange la pioche et tirer_carte qui permet de tirer une carte de la pioche, si la pioche est vide alors elle renvoie -1.

mecanique.c

Le module mecanique.c implémente les règles du jeu, notamment l'application des effets des cartes (appliquer_carte), la détection des doublons (verifier_doublon), le calcul des scores (compter_differentes) et la gestion des conditions de victoire ou de défaite pendant une manche (jouer_tour).

joueur.c

Le module joueur.c est dédié à la gestion des joueurs. Il permet leur création (connexion), leur initialisation en début de manche et le suivi de leurs scores au cours de la partie (init_joueur_manche).

affichage.c

Le module affichage.c s'occupe de toute la partie interface, notamment avec la génération et l'affichage des cartes (generer_lignes_cartes) , le plateau de jeu (afficher_plateau) ainsi qu'une fonction qui permet à l'utilisateur de taper sur entrée pour passer à la manche suivante ce qui permet de rendre le déroulement du jeu plus lisible et interactif pour l'utilisateur (attendre_entree).

fichier.c

Enfin, le module fichier.c permet la sauvegarde des résultats dans un fichier texte, incluant le nom et le score de chaque joueur (fichier_score). Il y a également la fonction `verif_carac` qui permet de vérifier si le nom du fichier donné par l'utilisateur est bien compatible.

3. Organisation de l'équipe

Dans un premier temps, en équipe, nous avons réalisé une carte mentale avec toutes les premières idées qu'on a eues. On a listé les fonctions et les structures. Ensuite on s'est réparti les fonctions. Lorsqu'une personne avait fini une tâche, on l'ajoutait au code commun. Inès s'est chargée de la gestion de la pioche, avec une amélioration (pioche spéciale) ainsi que de la création de fichier en fin de partie. Leila s'est chargée de l'interface et de la mécanique du jeu. Et enfin, Florentin s'est chargé de la logique principale du jeu avec le code principal, la gestion des joueurs et du déroulement de chaque manche et chaque tour.

Il y a également eu à la fin une vérification de la part de chaque membre de l'équipe de l'ensemble du code, afin de renforcer les conditions, d'éviter les erreurs et d'avoir un programme sécurisé.

4. Difficultés rencontrées

L'une des principales difficultés rencontrées durant le projet concernait l'affichage des cartes dans le terminal. Au départ, nous avons des difficultés à afficher correctement plusieurs cartes côte à côte, ce qui rendait l'interface peu lisible. Nous avons également rencontré des problèmes d'affichage avec certains caractères spéciaux et encadrés utilisés pour améliorer l'interface du jeu, notamment à cause des différences d'encodage selon les systèmes d'exploitation.

5. Solutions apportées

Pour résoudre le problème d'affichage des cartes, nous avons utilisé la fonction `sprintf` afin de générer chaque ligne des cartes dans des chaînes de caractères avant de les afficher. Cette méthode nous a permis d'aligner correctement plusieurs cartes les unes à côté des autres et d'obtenir un affichage plus propre dans le terminal. Concernant les problèmes liés aux caractères spéciaux, nous avons ajouté une initialisation spécifique de la console selon le système d'exploitation utilisé. Sous Windows, nous avons utilisé `SetConsoleOutputCP(CP_UTF8)` afin d'activer l'encodage UTF-8, tandis que sous Linux

nous avons utilisé `setlocale`. Cela a permis d'assurer un affichage correct des caractères spéciaux et des bordures utilisées dans l'interface.

6. Points non réussis

Malgré le bon fonctionnement global du projet, le programme aurait pu être davantage optimisé, notamment au niveau de la gestion de la mémoire sur certaines fonctions. Nous avons cependant choisi de privilégier une base de jeu stable, fonctionnelle et correctement sécurisée avant d'ajouter des améliorations plus avancées, comme les cartes spéciales « STOP », « 2nde chance » et « 3 à la suite », qui n'ont pas été implémentées, car elles auraient nécessité de repenser une partie importante de la logique du jeu. Le mode joueur contre ordinateur n'a pas non plus été développé pour ses mêmes raisons.